Python -String

Objective

- Basics of string
- String Slicing
- String Operators
- format() Methods
- Escape Sequence
- Built in String functions
- String methods

Basics of string

- Python string is the collection of the characters surrounded by single quotes, double quotes, or triple quotes.
- String is a sequence of characters.
- String may contain alphabets, numbers and special characters.
- Strings is immutable in nature.
- Each character is encoded in the ASCII or Unicode character. So we can say that Python strings are also called the collection of Unicode characters.
- **Example**

```
str="Welcome"
name='Snehal’
msg='''Hi Ram'''
msg1="""Hi Ram1"""
print(str, name,msg,msg1)
```


www.object.co.in

string creation:

We can create a string by enclosing the characters in single-quotes or double- quotes. Python also provides triple-quotes to represent the string, but it is generally used for multiline string or docstrings.

```
# all of the following are equivalent
my_string = 'Hello'
print(my_string)

my_string = "Hello"
print(my_string)

my_string = '''Hello'''
print(my_string)

# triple quotes string can extend multiple lines
my_string = """Hello, welcome to
        the world of Python"""
print(my_string)
```

Output:

```
Hello
Hello
Hello
Hello, welcome to
        the world of
Python
```


www.object.co.

String Slicing

- Slicing operation is used to return/select/slice the particular substring based on user requirements.
- A segment of string is called slice.
- Syntax: string_variablename [start:end]

str = "HELLO"

H	E	L	L	O
0	1	2	3	4

str[0] = 'H'

str[1] = 'E'

str[2] = 'L'

str[3] = 'L'

str[4] = 'O'

str[:] = 'HELLO'

str[0:] = 'HELLO'

str[:5] = 'HELLO'

str[:3] = 'HEL'

str[0:2] = 'HE'

str[1:4] = 'ELL'

str = “HELLO”

H	E	L	L	O
-5	-4	-3	-2	-1

str[-1] = ‘O’

str[-2] = ‘L’

str[-3] = ‘L’

str[-4] = ‘E’

str[-5] = ‘H’

str[-3:-1] = ‘LL’

str[-4:-1] = ‘ELL’

str[-5:-3] = ‘HE’

str[-4:] = ‘ELLO’

str[::-1] = ‘OLLEH’

www.object.co.in

Rules of Slicing

- We can access **individual characters using indexing** and a **range of characters using slicing**. Index starts from 0.
- Trying to access a character out of index range will raise an IndexError.
- The index must be an integer.
- We can't use floats or other types, this will result into TypeError.
- Python allows negative indexing for its sequences.
- The index of -1 refers to the last item, -2 to the second last item and so on.
  We can access a range of items in a string by using the slicing operator :(colon).

www.object.co.in

Accessing char in string

0	1	2	3	4	5	6	7	8	9	10
C	o	r	o	n	a	V	i	r	u	s
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

```
str = 'CoronaVirus'
print(str[-1])
print(str[-3])
print(str[-2:])
print(str[-4:-1])
print(str[-7:-2])
print(str[7:-2])
print(str[7:-5])
# Reversing the given string
print(str[::-1])
print(str[-12])
```

output:

```
s
r
us
iru
naVir
ir

suriVanoroC
Traceback (most recent call last):
  File
"C:/Users/sneha/OneDrive/Desktop/
demo1.py", line 11, in <module>
    print(str[-12])
IndexError: string index out of range
```


www.object.co.in

Reassigning Strings

- Updating the content of the strings is as easy as assigning it to a new string.
- The string object doesn't support item assignment i.e., A string can only be replaced with new string since its content cannot be partially replaced.
- Strings are immutable in Python.

Example 1

```
str = "HELLO"
str[0] = "h"
print(str)

Output:
Traceback (most recent call last):
  File "12.py", line 2, in <module>
    str[0] = "h";
TypeError: 'str' object does not support item assignment
```

However, in example 1, the string str can be assigned completely to a new content as specified in the following example.

Example 2

```
str = "HELLO"
print(str)
str = "hello"
print(str)
```

Output:

```
HELLO
hello
```

Deleting the String

- As we know that strings are immutable. We cannot delete or remove the characters from the string. But we can delete the entire string using the **del keyword**.

```
str = "Welcome"
del str[1]
```

Output:

```
TypeError: 'str' object doesn't support item deletion
```

Now we are deleting entire string.

```
str1 = "Welcome"
del str1
print(str1)
```

Output:

```
NameError: name 'str1' is not defined
```


www.object.co.in

Iteration

The for loop is used to iterate over the string elements.

```
s="Ramayan"
for x in s:
    print(x)
```

output:

```
R
a
m
a
y
a
n
```


www.object.co.in

String Operators

Assume string variable a holds 'Hello' and variable b holds 'Python', then –

- **+ Concatenation -** Adds values on either side of the operator Example: a + b will give HelloPython
- *** Repetition -** Creates new strings, concatenating multiple copies of the same string Example:a*2 will give -HelloHello
- **[] Slice** - Gives the character from the given index . Example:a[1] will give e
- **[:] Range Slice -** Gives the characters from the given range Example: a[1:4] will give ell
- **in Membership** - Returns true if a character exists in the given string Example: H in a will give 1
- **not in Membership** - Returns true if a character does not exist in the given string Example: M not in a will give 1
- **% Format** - Performs String formatting

www.object.co.in

String comparision

- We can compare two strings using comparision operators such as ==, !=, <,<=,>, >=
- Python compares strings based on their corresponding ASCII values.

Example of string comparision

```
str1="green"
str2="black"
print("Is both Equal:", str1==str2)
print("Is str1> str2:", str1>str2)
print("Is str1< str2:", str1<str2)
```

OUTPUT:

```
Is both Equal: False
Is str1> str2: True
Is str1< str2: False
```


www.object.co.in

String Formatting Operator

print ("My name is %s and weight is %d kg!" % ('Zara', 21))

Output:

My name is Zara and weight is 21 kg!

Here is the list of complete set of symbols which can be used along with % -

sr no	Format	Conversion
1	%c	character
2	%s	string conversion via str() prior to formatting
3	%i	signed decimal integer
4	%d	signed decimal integer
5	%u	unsigned decimal integer
6	%f	float value

www.object.co.in

format() Methods

- The format() method that is available with the string object is very versatile and powerful in formatting strings.
- Format strings contain curly braces {} as placeholders or replacement fields which get replaced.
- We can use positional arguments or keyword arguments to specify the order.

```
# Python string format() method
# default(implicit) order
default_order = "{}, {} and {}".format('John','Bill','Sean')
print(' --- Default Order ---')
print(default_order)
```

output:

```
--- Default Order ---
John, Bill and Sean
```


www.object.co.in

```
# order using positional argument
positional_order = "{1}, {0} and {2}".format('John','Bill','Sean')
print(' --- Positional Order ---')
print(positional_order)
```

Output:

```
--- Positional Order ---
Bill, John and Sean
```

```
# order using keyword argument
keyword_order = "{s}, {b} and {j}".format(j='John',b='Bill',s='Sean')
print(' --- Keyword Order ---')
print(keyword_order)
```

Output:

```
--- Keyword Order ---
Sean, Bill and John
```


www.object.co.in

f function()

OBJECT™ TECHNOLOGIES

In 3.8.5 version we have f function()

```
name = "Rama"
age = 74
print(f"Hello, {name}. You are {age}. {3*2}")
```

Output:

Hello, Rama. You are 74. 6

Escape Sequence

- The backslash() symbol denotes the escape sequence.
- The backslash can be followed by a special character and it interpreted differently.
- The single quotes inside the string must be escaped. We can apply the same as in the double quotes.

T

Escape Character	Meaning
\’	Single quote
\”	Double quote
\\	backslash
\n	New line
\r	Carriage Return
\t	Horizontal tab
\b	Backspace

www.object.co.in

Built in String functions

Python mainly contains 3 inbuilt string functions.

1. len()- Find out the length of characters in string
2. min()- Smallest value in a string based on ASCII values
3. max()- Largest value in a string based on ASCII values

Example:

```
mystr="Welcome"
print(len(mystr))
print(min(mystr))
print(max(mystr))
```

Output:

```
7
W
o
```


www.object.co.in

String Methods

OBJECT™ TECHNOLOGIES

Cases II

s.casefold()	Casefold s (aggressive lowercasing for caseless matching) # 'ßorat' => 'ssorat'
s.islower()	Return true if s is lowercase
s.istitle()	Return true if s is titlecased # 'Hello World' => true
s.isupper()	Return true if s is uppercase

s.capitalize()	Capitalize s # 'hello' => 'Hello'
s.lower()	Lowercase s # 'HELLO' => 'hello'
s.swapcase()	Swap cases of all characters in s # 'Hello' => "hELLO"
s.title()	Titlecase s # 'hello world' => 'Hello World'
s.upper()	Uppercase s # 'hello' => 'HELLO'

Whitespace I

s.center(width)	Center s with blank padding of width # 'hi' => ' hi '
s.isspace()	Return true if s only contains whitespace characters
s.ljust(width)	Left justifiy s with total size of width # 'hello' => 'hello '
s.rjust(width)	Right justify s with total size of width # 'hello' => ' hello'
s.strip()	Remove leading and trailing whitespace from s # ' hello ' => 'hello'

String methods

OBJECT™ TECHNOLOGIES

Find / Replace I

s.index(s2, i, j)	Index of first occurrence of s2 in s after index i and before index j
s.find(s2)	Find and return lowest index of s2 in s
s.index(s2)	Return lowest index of s2 in s (but raise ValueError if not found)
s.replace(s2, s3)	Replace s2 with s3 in s
s.replace(s2, s3, count)	Replace s2 with s3 in s at most count times
s.rfind(s2)	Return highest index of s2 in s
s.rindex(s2)	Return highest index of s2 in s (raise ValueError if not found)

Inspection I

s.endswith(s2)	Return true if s ends with s2
s.isalnum()	Return true if s is alphanumeric
s.isalpha()	Return true if s is alphabetic
s.isdecimal()	Return true if s is decimal
s.isnumeric()	Return true if s is numeric
s.startswith(s2)	Return true is s starts with s2

Example:

```
name='''coronaviruscoronavirus'''

print (name[::-1])
print(name.capitalize())
print(name.upper())
print(name.lower())
print(name.title())
print(name.count('c'))
print(name.replace('c','C'))
print(name.isalpha())
print(name.startswith('c'))
print(name.find('v'))
print(name.index('s'))
print(name.rfind('v'))
print(name.rindex('s'))
```

- **Output**:

```
surivanorocsurivanoroc
Coronaviruscoronavirus
CORONAVIRUSCORONAVI
RUS
coronaviruscoronavirus
Coronaviruscoronavirus
2
CoronavirusCoronavirus
True
True
6
10
17
21
```

Summary

- Basics of string
- String Slicing
- String Operators
- format() Methods
- Escape Sequence
- Built in String functions
- String methods

Thank You